TU WIEN Informatics

COMPUTATIONAL SUSTAINABILITY RESEARCH UNIT

Data-Intensive Computing

2.0 VU / 3.0 ECTS, 2026S

Lecture 3 - Basics of NLP with Apache Hadoop + Assignment 1

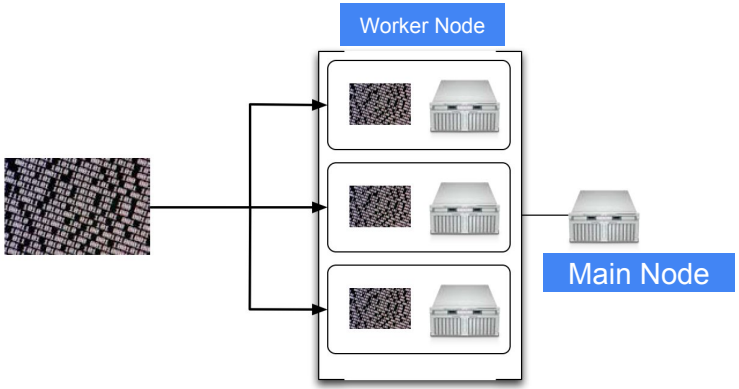
Daniel Hofstätter
data-intensive-computing@tuwien.ac.at

Credits for the original version of slides to P. Knees and D. Winkler

Recap

Core Ideas: Data Locality

- **Bring code to data rather than data to code**
- Design goal: maximize disk I/O
- Scale out (more machines) rather than scale up (bigger machines)
- Seeking too slow – go through entire data set in one (or a small number) of passes
- Nodes talk to each other as little as possible



MapReduce Job Execution (simplified)

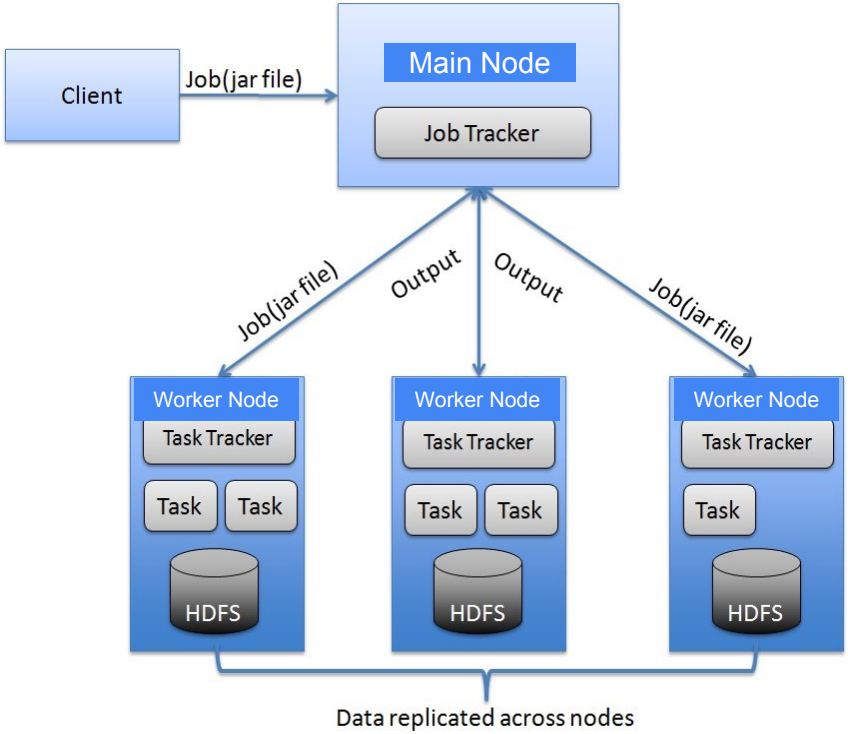


Image source: R. S. Ramanujam, R. Nancyamala, J. Nivedha and J. Kokila, Sentiment analysis using big data, (2015)

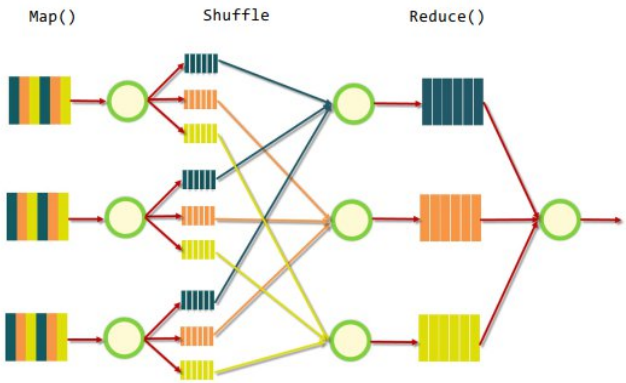
- **JobTracker** on Master Node
- **TaskTracker** on each worker node
- Output written locally on worker nodes

- Job phases:
  - **Map**
  - **Shuffle & Sort**
  - **Reduce**

MapReduce Model

Basic process structure:

1. Map function is applied to every input key-value pairs (record)
2. Map function generates intermediate key-value pairs
3. Intermediate key-values are sorted and grouped by key
4. Reduce is applied to sorted and grouped intermediate key-values
5. Reduce emits result key-values



Map function:

$(k_1, v_1) \rightarrow [(k_2, v_2)]$

Reduce function:

$(k_2, [v_2]) \rightarrow [(k_3, v_3)]$

[] .. list

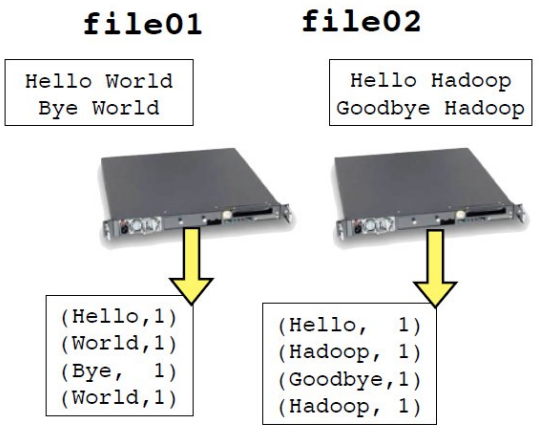
MapReduce Word Count in Pseudo-code

```
class Mapper
    method Map(docid a, doc d)
        for all term t ∈ doc d do
            Emit(term t, count 1)
```

```
class Reducer
    method Reduce(term t, counts [c1, c2, ...])
        sum ← 0
        for all count c ∈ counts [c1, c2, ...] do
            sum ← sum + c
        Emit(term t, count sum)
```


Image Source: J. Lin and C. Dyer, *Data-intensive text processing with MapReduce*. [San Rafael, Calif.]: Morgan & Claypool Publishers, 2010.

Word Count: MapReduce Execution (1)



Map

Image source: https://github.com/ljdursi/hadoop-for-hpcers-tutorial/blob/master/presentation/images2/DD-Network.png

Word Count: MapReduce Execution (2)

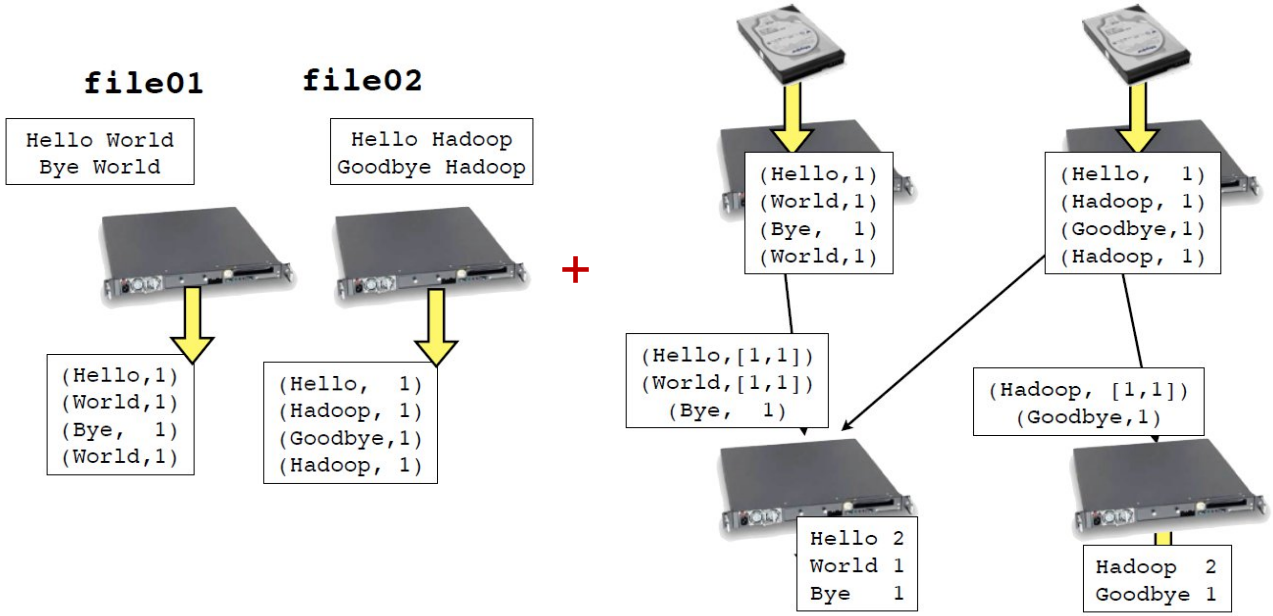


Image source: https://github.com/ljdursi/hadoop-for-hpcers-tutorial/blob/master/presentation/images2/DD-Network.png

Word Count: MapReduce Execution (3)

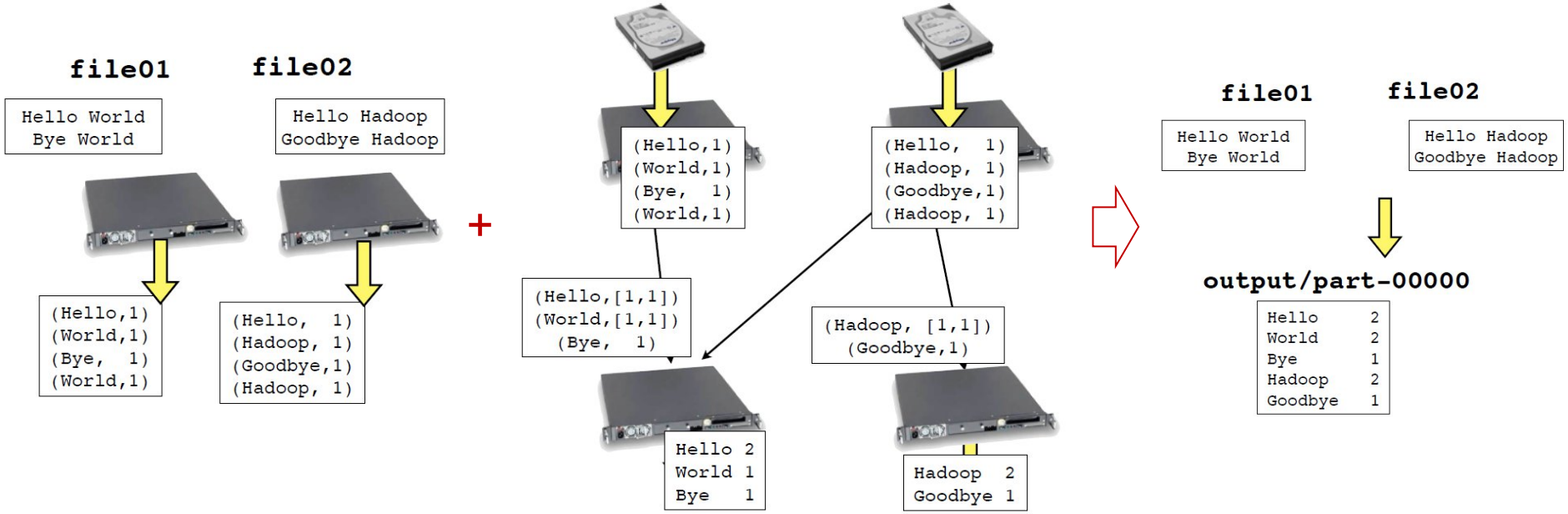


Image source: https://github.com/ljdursi/hadoop-for-hpcers-tutorial/blob/master/presentation/images2/DD-Network.png

MapReduce Application Ingredients

- Primarily Java-based
- We will focus on the Python version of the library (mrjob) although Java implementations tend to perform better
  - Assumes more familiarity with Python
  - Mrjob allows local testing before feeding to Hadoop
  - Jobs can be built in a single class
- Components
  - **Mapper**
  - **Reducer**
  - Combiner
  - Partitioner

Word Count Example: Mapper

```python
from mrjob.job import MRJob
class MRWordCount(MRJob):
    def mapper(self, _, line):
        for word in line.split():
            yield(word, 1)
    def reducer(self, word, counts):
        yield(word, sum(counts))
if __name__ == '__main__':
    MRWordCount.run()
```

Word Count Example: Reducer

```python
from mrjob.job import MRJob
class MRWordCount(MRJob):
    def mapper(self, _, line):
        for word in line.split():
            yield(word, 1)
    def reducer(self, word, counts):
        yield(word, sum(counts))
if __name__ == '__main__':
    MRWordCount.run()
```

Observations: Actual code

Mapper:

```
def mapper(self, _, line):
    for word in line.split():
        yield(word, 1)
```

- Get the string out of the Text value
- Tokenize it (split it by spaces)
- For each token,
- emit (word,one)

Reducer:

```
def reducer(self, word, counts):
    yield(word, sum(counts))
```

- Operates on a single (k,[v]).
- Loop over values and sum them up
- emit(key, sum)

Optional: Combiner

- Collates the output of each mapper individually before feeding to reducers
- Run by each mapper on the output of the mapper, before its fed to the shuffle.
- In this case, the combiner is the same as the reducer
- Useful if many mappers feed into a reducer – reduces the load on the reducer
- Not all problems lend themselves to the obvious use of a combiner, and it is not necessarily identical to the reducer.
- If reducer is commutative and associative ((k2,[v2]) → (k2,v2)), can use reducer as the combiner

```python
from mrjob.job import MRJob
import re

WORD_RE = re.compile(r"[\w']+")


class MRWordFreqCount(MRJob):

    def mapper(self, _, line):
        for word in WORD_RE.findall(line):
            yield word.lower(), 1

    def combiner(self, word, counts):
        yield word, sum(counts)

    def reducer(self, word, counts):
        yield word, sum(counts)


if __name__ == '__main__':
    MRWordFreqCount.run()
```

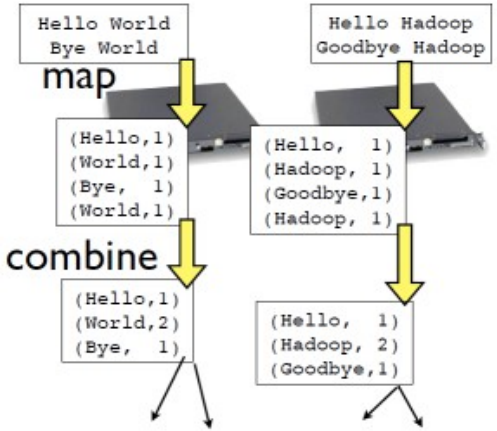


Image: https://github.com/ljdursi/hadoop-for-hpcers-tutorial/

Testing Locally vs Working on Hadoop

- Local testing with MrJob
  - Allows testing of code without submitting to Hadoop cluster
  - Not as efficient (for large files) but debug code faster!
  - The difference is in the terminal command
  - REMEMBER: Test with a smaller input file!

```
python mrjob_job.py input.txt > output.txt
```

vs

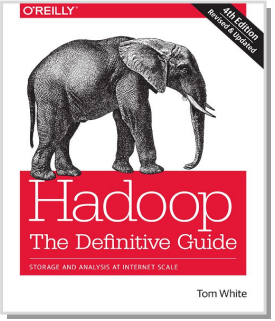
```
python mrjob_job.py --hadoop-streaming-jar /usr/lib/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -r hadoop
input.txt > output.txt
```

Reference: https://mrjob.readthedocs.io/en/latest/guides/testing.html

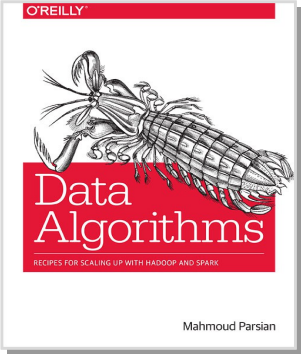
Wrap-up: MapReduce Job Design

1. Divide task into map and reduce parts
   Hint: Often easier to work your way backwards from the reducer
2. <k,v> design:
   - What are the input splits that go into the Mappers?
   - Do I need a custom InputFormat?
   - Do I need custom DataTypes/Avro schemas..?
   - What are the <k,v> pairs emitted by the Mapper and consumed by the Reducers?
   - What <k,v> pairs need to be emitted by the Reducer?
   - Do I need to include data in the keys for secondary sorting?
3. Can I use a custom combiner, partitioner?
4. Do I need to chain multiple jobs or can I fit the task into one?

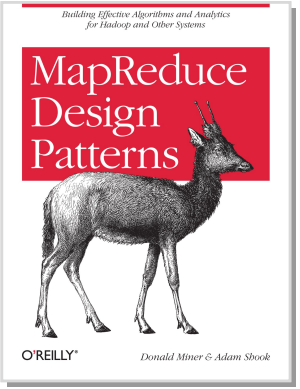
Resources



White, Tom. *Hadoop: The definitive guide*", 4th ed. O'Reilly Media, Inc.", 2015.



Parsian, Mahmoud. *Data Algorithms: Recipes for Scaling Up with Hadoop and Spark*. " O'Reilly, 2015.

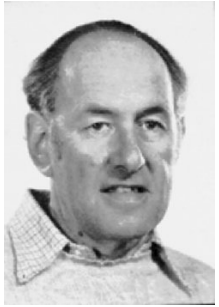


Miner, Donald, and Adam Shook. *MapReduce design patterns: building effective algorithms and analytics for Hadoop and other systems*. " O'Reilly Media, Inc.", 2012.

Basics of NLP (Natural Language Processing)

Information Retrieval

- Information and data explosion on Web and in specialised domains
- “Information retrieval is a field concerned with the structure, analysis, organization, storage, searching, and retrieval of information.” (Salton, 1968)
- General definition that can be applied to many types of information and search applications
- Primary focus of IR since the 50s has been on text and documents



Slide based on prev AIR course

Textual Data Sources for Information Retrieval

- Books
- Newspapers
- Scientific articles
- Patents
- Web pages
- Blogs
- eMails
- Product reviews (e.g. Amazon, Epinions.com)
- Social media: microblogs, status messages, …
- RSS feeds
- User tags (e.g. Flickr)

Outline: Text Processing

- Basic Concepts
  - Document, Corpus
  - Tokens, N-grams, and Terms
  - Bag-of-Words (BoW)
  - Stop-word removal (aka “stopping”)
  - Case-folding
  - Stemming
  - Vector Space Model
- Text Similarity and Retrieval
- Text Categorization
  - Term selection

Basic concepts: Document and Corpus

- A **document** is a self-contained unit of text (including structural elements such as HTML or XML tags) which can be returned as a search result
- A set of documents belonging together is referred to as **corpus**
- Corpus of web pages, eMails, songs, etc.

- This document-centric view is also used in all other domains, e.g., an image (music piece, artist, video, etc.) is also considered a document that can be retrieved

Basic concepts: Tokens, N-grams, Terms

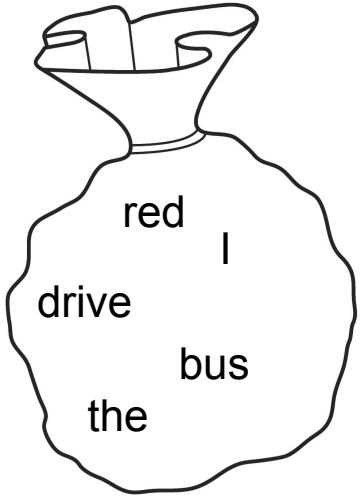
- **Tokens**: sequences of alphanumeric characters, given as result of a tokenization process (e.g. splitting a text at whitespaces, non-alphanumeric characters, or at HTML tags; dependent on corpus; confines the class of queries that the retrieval system can deal with)

- **N-gram**: sequence of N consecutive tokens appearing in a document (unigram, bigram, 3-gram, …, N-gram)

- **Index Terms** (or just “terms”): set of tokens or N-grams that are included in the index (i.e. data structure to store the corpus for IR purposes)

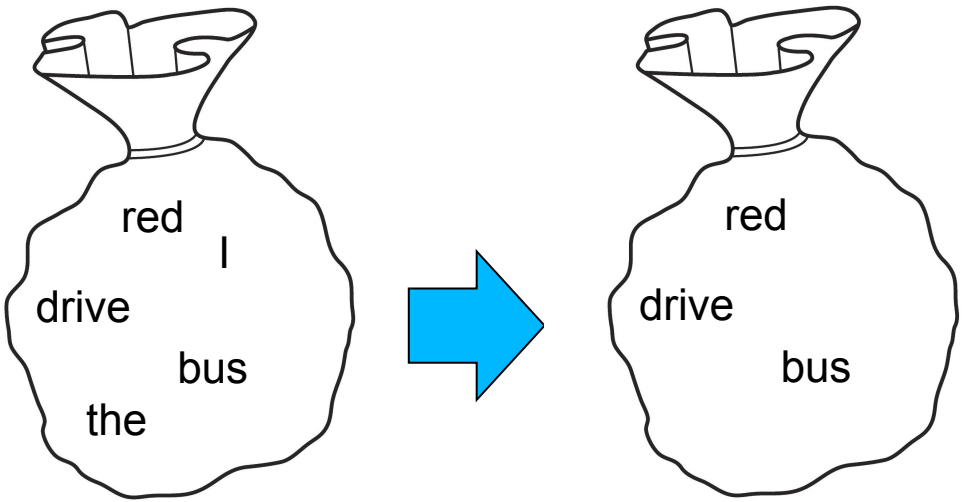
Basic concepts: Bag of Words (BoW)

- First, tokenization: split up sequence of tokens for representing text documents
- Each document is represented as an unordered set of terms (*bag*)
- Sequence of terms in a document
  is not considered important
  ➡ such features foremost describe *text vocabulary*



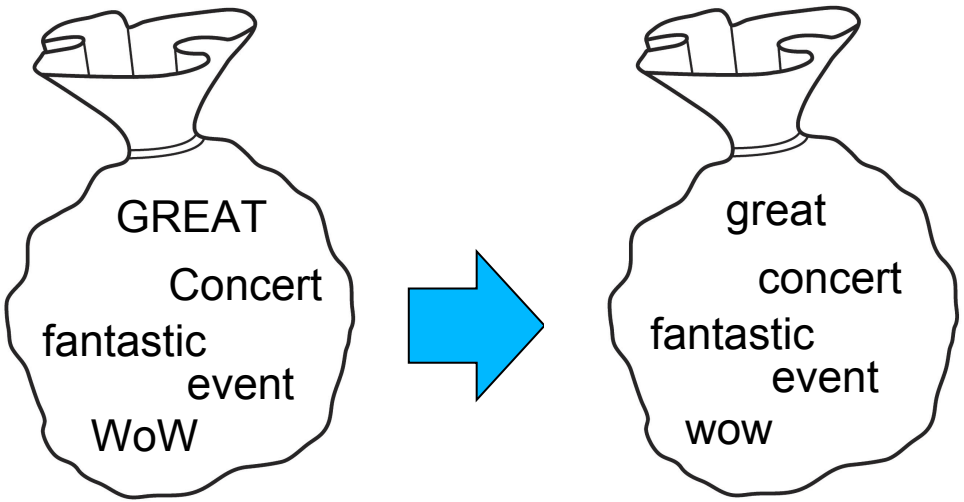
Basic concepts: Stopping

- Remove terms that are not discriminative for documents
- Some typical stop words:
  I, a, about, an, are, as, at, be, by, com, de, en, for, from, how, in, is, it, la, of, on, or, that, the, this, to, was, what, when, where, who, will, with, und, the, www
- Rationale: Reducing computational and space complexity during indexing and retrieval



Basic concepts: Case-folding

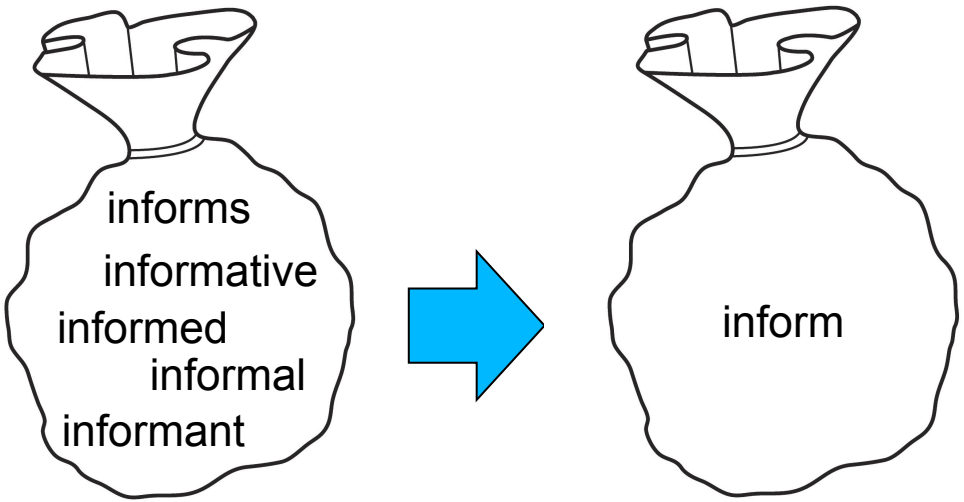
- Convert all characters to lower-case (done for queries and documents before stored in index)
- Rationale: Match queries and documents, irrespective of capitalization of particular terms



Not always a good idea though: US vs us?

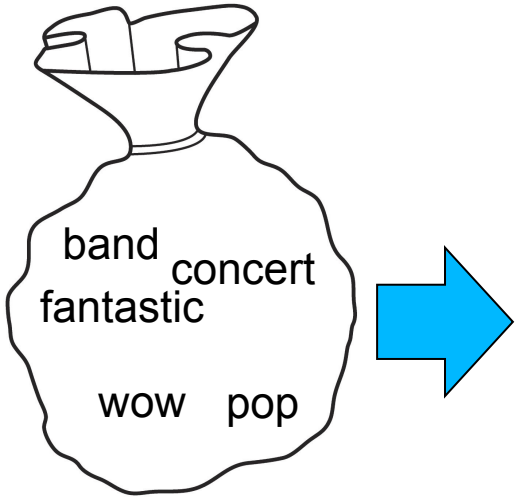
Basic concepts: Stemming

- Reduce all words to their linguistic stem, i.e. root form (done during indexing and querying)
- Rationale: match queries and documents through terms of same root meaning, e.g. the query “runs” would match the occurrence of “running” in an indexed document (by their common stem “run”)
- Porter stemmer for English (Porter 1980)
- More general/elaborated: lemmatization



Basic concepts: Vector Space Model (VSM)

- Represent each document by a vector in a high-dimensional feature space (dimensionality = cardinality of index term set).
- Typically, each dimension corresponds to the weight given to the respective term in the index term set, e.g., how often does the term occur in the doc.
- Index terms = [great, wow, pop, concert, band, event, fantastic]



BoW for document *d*

Term	Weight
great	0.0
wow	2.36
pop	0.46
concert	0.82
band	1.03
event	0.0
fantastic	1.42

term weight vector of *d*

Term weighting

- Crucial for VSM-based retrieval
- Assign a weight (an importance score) to each term t in each document d
- How to compute the weight? → three monotonicity assumptions
- Most prominent term weighting function is term frequency by inverse document frequency (tf-idf)

$\mathrm{tf}_{d,t}$ ... term frequency, how often term t occurs in document d

df_t ... document frequency, number of documents containing term t

$\mathrm{idf}_t = \log \frac{N}{\mathrm{df}_t}$ ... inverse document frequency for term t and number of documents N

$(\mathrm{tf} - \mathrm{idf})_{t,d} = \mathrm{tf}_{d,t} \times \mathrm{idf}_t$

Term weighting: Monotonicity Assumptions

1. **Multiple appearances of a term in a document are no less important than single appearances (TF assumption)**
   Importance of a term for a document d is the higher, the more often it appears in d.
2. **Rare terms are no less important than frequent ones (IDF assumption)**
   Importance of a term is the higher, the more rarely it appears among all documents (i.e. in the corpus).
3. **Long documents are no more important than short documents (normalization assumption)**
   normalization by document/query length; usually performed in similarity computation (**cosine measure**) between q and d.

Term weighting: TF and IDF

Denominations for variables (used in various TF and IDF variants)

$\mathcal{D}$	set of documents
N	number of documents
$f_{d,t}$	number of occurrences of term t in document d
f_t	number of documents containing term t
F_t	total number of occurrences of t in the collection
$\mathcal{T}_d$	the set of distinct terms in document d
f_d^m	the largest $f_{d,t}$ of all terms t in d
f^m	the largest f_t in the collection
$r_{d,t}$	term frequency, see Table II
w_t	inverse document frequency, see Table III
W_d	document length of d

Term weighting: TF (various formulations)

Abbr.	Description	Formulation
TF_A	Formulation used for binary match SB = b	$r_{d,t} = \begin{cases} 1 & \text{if } t \in \mathcal{T}_d \\ 0 & \text{otherwise} \end{cases}$
TF_B	Standard formulation SB = t	$r_{d,t} = f_{d,t}$
TF_C	Logarithmic formulation	$r_{d,t} = 1 + \log_e f_{d,t}$
TF_C2	Alternative logarithmic formulation suited for $f_{d,t} < 1$	$r_{d,t} = \log_e(1 + f_{d,t})$
TF_C3	Alternative logarithmic formulation as used in *ltc* variant	$r_{d,t} = 1 + \log_2 f_{d,t}$
TF_D	Normalized formulation	$r_{d,t} = \frac{f_{d,t}}{f_d^m}$
TF_E	Alternative normalized formulation. Similar to [55] we use $K = 0.5$. SB = n	$r_{d,t} = K + (1 - K) \cdot \frac{f_{d,t}}{f_d^m}$
TF_F	Okapi formulation, according to [55, 36]. For W we use the vector space formulation, i.e., the Euclidean length.	$r_{d,t} = \frac{f_{d,t}}{f_{d,t} + W_d / av_{d \in D}(W_d)}$
TF_G	Okapi BM25 formulation, according to [35].	$r_{d,t} = \frac{(k_1+1) \cdot f_{d,t}}{f_{d,t} + k_1 \cdot \left[(1-b) + b \cdot \frac{W_d}{av_{d \in D}(W_d)} \right]}$ $k_1 = 1.2, b = 0.75$

Term weighting: IDF (various formulations)

Abbr.	Description	Formulation
IDF_A	Formulation used for binary match SB $= x$	$w_t = 1$
IDF_B	Logarithmic formulation SB $= f$	$w_t = \log_e \left(1 + \frac{N}{f_t}\right)$
IDF_B2	Logarithmic formulation used in *ltc* variant	$w_t = \log_e \left(\frac{N}{f_t}\right)$
IDF_C	Hyperbolic formulation	$w_t = \frac{1}{f_t}$
IDF_D	Normalized formulation	$w_t = \log_e \left(1 + \frac{f_m}{f_t}\right)$
IDF_E	Another normalized formulation SB $= p$	$w_t = \log_e \frac{N - f_t}{f_t}$
	The following definitions are based on the term's noise n_t and signal s_t.	$n_t = \sum_{d \in \mathcal{D}_t} \left(-\frac{f_{d,t}}{F_t} \log_2 \frac{f_{d,t}}{F_t}\right)$ $s_t = \log_2(F_t - n_t)$
IDF_F	Signal	$w_t = s_t$
IDF_G	Signal-to-Noise ratio	$w_t = \frac{s_t}{n_t}$
IDF_H		$w_t = \left(\max_{t' \in \mathcal{T}} n_{t'}\right) - n_t$
IDF_I	Entropy measure	$w_t = 1 - \frac{n_t}{\log_2 N}$
IDF_J	Okapi BM25 IDF formulation, according to [35, 31]	$w_t = \log \frac{N - f_t + 0.5}{f_t + 0.5}$

Term weighting

- Very frequently used tf-idf function (ltc variant):

TF_C x IDF_B2

$$tfidf_{t,d} = (1 + \log(f_{d,t})) \quad \log(\frac{N}{f_t})$$

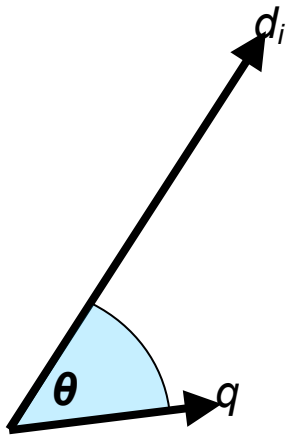
- Why logarithm? Dampens very high frequencies of only a few words (cf. “Zipf’s law”)

Choice of term weighting function highly depends on the corpus under consideration (e.g. in short texts tf is almost worthless – status messages, microblogs, etc.).

Similarity measurement

- Need to assess the match between a query q and each document d
- Quantifies the correlation between q and d
- Very prominent similarity measure: cosine similarity

$$sim(q,d_i) = \cos\theta = \frac{\vec{d}_i \cdot \vec{q}}{\left|\vec{d}_i\right| \times \left|\vec{q}\right|}$$



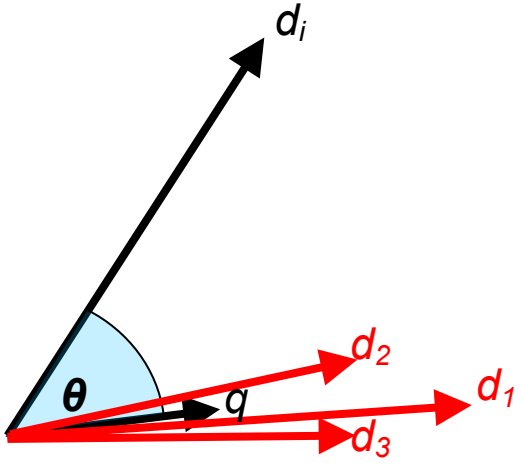
- Implicit normalization (ignores document length)

Choice of similarity function highly depends on retrieval task and corpus under consideration.

Retrieval model: Vector Space Model

- Compute similarity between query *q* and each document d_i based on *q*'s and d_i's term weighting vectors
- Return an ordered list of documents, ordered by $sim(q,d_i)$
  ➡ **Retrieval result!**

$$sim(q,d_i) = \cos\theta = \frac{\vec{d}_i \cdot \vec{q}}{|\vec{d}_i| \times |\vec{q}|}$$



- Result in example for *q*:
  1. d_1
  2. d_2
  3. d_3

Other common similarity measures

Abbr.	Description	Formulation
SIM_INN	Inner Product	$S_{d_1,d_2} = \sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})$
SIM_COS	Cosine Measure	$S_{d_1,d_2} = \frac{\sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})}{W_{d_1} \cdot W_{d_2}}$
SIM_DIC	Dice Formulation	$S_{d_1,d_2} = \frac{2 \sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})}{W_{d_1}^2 + W_{d_2}^2}$
SIM_JAC	Jaccard Formulation	$S_{d_1,d_2} = \frac{\sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})}{W_{d_1}^2 + W_{d_2}^2 - \sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})}$
SIM_OVL	Overlap Formulation	$S_{d_1,d_2} = \frac{\sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} \cdot w_{d_2,t})}{\min(W_{d_1}^2, W_{d_2}^2)}$
SIM_EUC	Euclidean Similarity	$D_{d_1,d_2} = \sqrt{\sum_{t \in \mathcal{T}_{d_1,d_2}} (w_{d_1,t} - w_{d_2,t})^2}$ $S_{d_1,d_2} = \left(\max_{d'_1,d'_2} (D_{d'_1,d'_2})\right) - D_{d_1,d_2}$
SIM_JEF	Jeffrey Divergence-based Similarity	$S_{d_1,d_2} = \left(\max_{d'_1,d'_2} (D_{d'_1,d'_2})\right) - D_{d_1,d_2}$ $D(F,G) = \sum_i \left(f_i \log \frac{f_i}{m_i} + g_i \log \frac{g_i}{m_i}\right)$ $m_i = \frac{f_i + g_i}{2}$

Back to Big Data Processing

and back to MapReduce…

- What do we need to calculate tf-idf for each document?
  - Counting how often a word occurs in a document (tf)
  - Counting how many documents contain a word (df)
  - The number of documents (N)

All of these can be obtained in a distributed fashion if we choose the (Key, Value) pairs in the WordCount example efficiently.

Text Classification

- Text retrieval built on feature design and similarity function to find documents similar to query
- In classification, we are interested in categorizing a text document into n classes
- Examples:
  - Email classification into ham vs. spam
  - Classifying news articles (eg. politics, sports, economy)
  - Categorization of writing style (eg. expository, descriptive, persuasive, narrative)

Text Classification

- Essentially a machine-learning task
- Wide variety of classification algorithms applicable: Naïve Bayes, k-NN, SVM, etc.
- tf-idf vectors serve often as document features

- Feature selection crucial
- Find most informative attributes (terms) for classification

Text Feature Selection

- Methods for “aggressive” dimensionality reduction
  - Text features typically high dimensional (all terms occurring in the corpus!)
  - Limit to most discriminative wrt. class labels
- Some common methods for text
  - DF Thresholding
  - Term Strength (based on clustering of documents)
  - Information Gain
  - Mutual Information
  - χ^2 **Statistic**

Yang, Y. and Pedersen, J.O.. "A comparative study on feature selection in text categorization." *ICML*. vol. 97. no. 412-420. 1997.

Pearson's χ^2 Statistic (chi-square)

- χ^2 test measures dependence between categorical stochastic variables (in our case *t* … term and *c* … category)
- $\chi^2_{t,c}$ value …lack of independence of term t from category c

$$\chi^2_{tc} = \frac{N(AD - BC)^2}{(A+B)(A+C)(B+D)(C+D)}$$

A … number of documents in c which contain t
B … number of documents not in c which contain t
C … number of documents in c without t
D … number of documents not in c without t
N … total number of retrieved documents (can be omitted for ranking)

2-way contingency table

	t	¬ t
c	A	C
¬ c	B	D

Pearson's χ^2 Statistic (chi-square)

- H_0 of χ^2 test: no relationship exists = categorical variables in population are independent
  - ~ for each cell: "deviation of observed from expected"
  - Test statistic: sum of all cells
  - Under H_0: test statistic follows approx. χ^2 dist. with one degree of freedom
- Sufficiently large test statistic ➡ reject H_0
- Test statistic is directly used as goodness criteria for separation
- Rank according to statistic for term selection

An Example of Chi-Square Ranked Terms

100	*sabbath	26	heavy	17	riff	12	butler
97	*pantera	26	ulrich	17	leaf	12	blackened
89	*metallica	26	vulgar	17	superjoint	12	bringin
72	*leppard	25	megadeth	17	maiden	12	purple
58	metal	25	pigs	17	armageddon	12	foolin
56	hetfield	24	halford	17	gillan	12	headless
55	hysteria	24	dio	17	ozzfest	12	intensity
53	ozzy	23	reinventing	17	leps	12	mob
52	iommi	23	lange	16	slayer	12	excitable
42	puppets	23	newsted	15	purify	12	ward
40	dimebag	21	leppards	15	judas	11	zeppelin
40	anselmo	21	adrenalize	15	hell	11	sandman
40	pyromania	21	mutt	15	fairies	11	demolition
40	paranoid	20	kirk	15	bands	11	sanitarium
39	osbourne	20	riffs	15	iron	11	*black
37	*def	20	s&m	14	band	11	appice
34	euphoria	20	trendkill	14	reload	11	jovi
32	geezer	20	snowblind	14	bassist	11	anger
29	vinnie	19	cowboys	14	slang	11	rocked
28	collen	18	darrell	13	wizard	10	drummer
28	hammett	18	screams	13	vivian	10	bass
27	bloody	18	bites	13	elektra	9	rocket
27	thrash	18	unforgiven	13	shreds	9	evil
27	phil	18	lars	13	aggression	9	loud
26	lep	17	trujillo	13	scar	9	hard

100 most discriminative terms for Heavy Metal texts

(score normalized to max. 100)

[Knees et al., 04]

Back to Big Data Processing

and back to MapReduce…

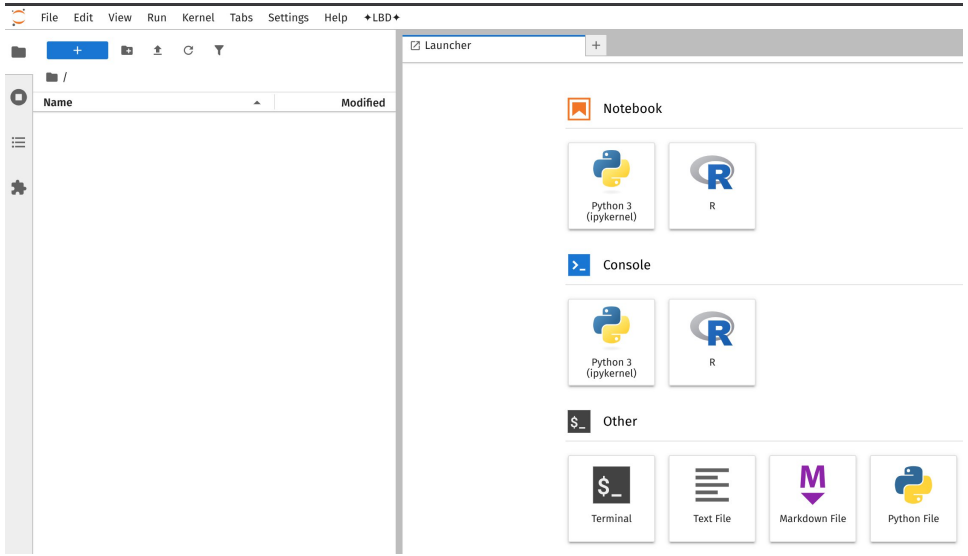
- What do we need to use χ^2 term selection?
  - A, B, C, D for each combination of *t*, *c*
  - Sorting according to resulting value

Again, can be obtained in a distributed fashion if we choose the (Key, Value) pairs in the WordCount example efficiently…

Assignment 1

Assignment 1: Using jupyter hub to develop with mrjob

- Go to https://lbd.tuwien.ac.at/ jupyter hub
  - Use TU Wien SSO to sign in
  - Needs VPN when using external network connection
- Usable as simple IDE
  - Running your code on cluster needs to be triggered from shell
  - You can access shell from jupyter hub launcher



Assignment 1: Instructions

Let’s have a look!

https://tuwel.tuwien.ac.at/mod/resource/view.php?id=2923395

Next steps…

- Revisit WordCount example from Exercise 0: contains principle structure of MapReduce program
- Consider the structure of the information you need (and when) and don’t need
- Tom White. Hadoop: The definitive guide, 4th ed., O'Reilly Media, Inc., 2015.

and/or:

- mrjob Documentation: https://mrjob.readthedocs.io/en/latest/